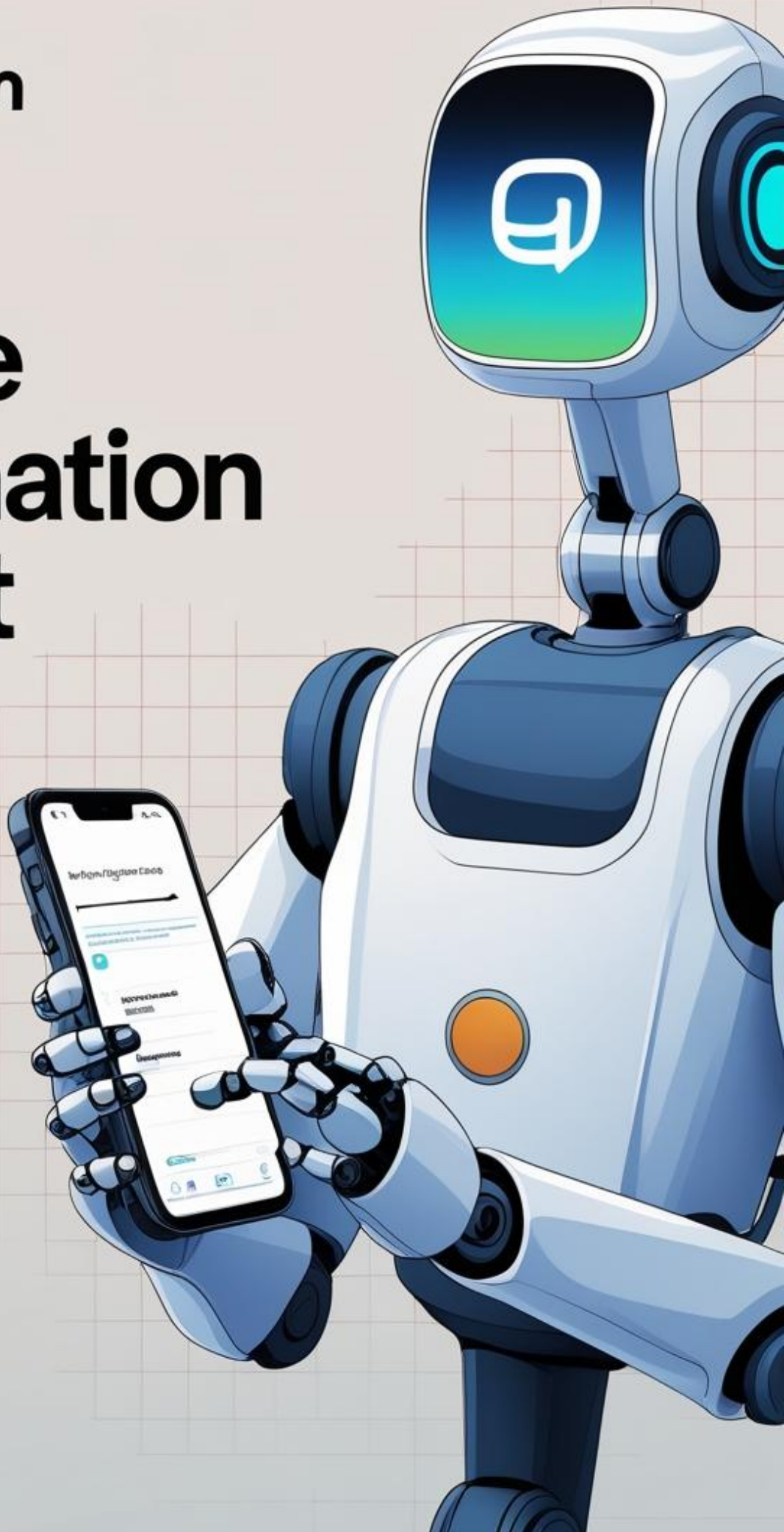




Mobile Automation Report



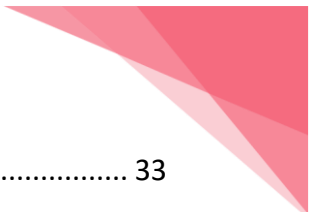


Appium Mobile Automation Testing Report



Contents

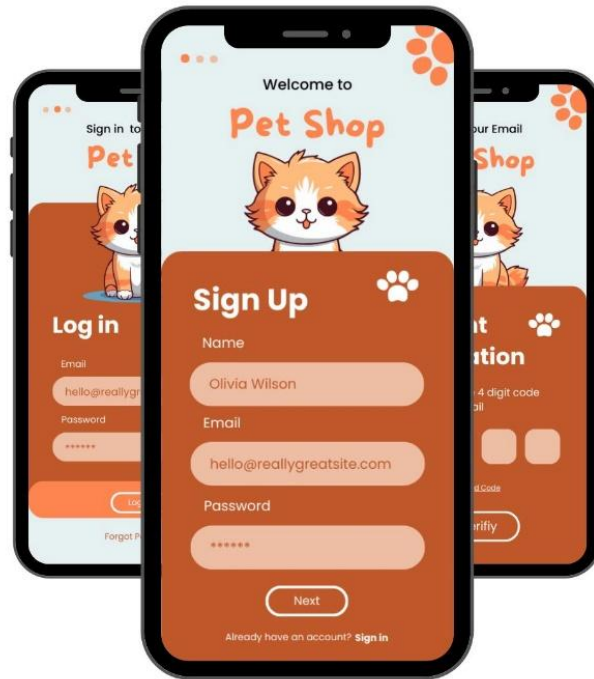
1. Introduction to Appium	5
2. What is Appium?	6
3. Why Use Appium?	7
4. Appium Architecture	8
4.1 Appium Client	8
4.2 Appium Server	8
4.3 Vendor Drivers	9
4.4 Desired Capabilities	9
5. Tools and Technologies	9
6. Step-by-Step Setup and Installation	9
6.1 Java Installation (Detailed)	10
6.2 Node.js and Appium Installation	11
6.3 Android Studio Setup	13
6.4 Appium Inspector Setup	15
7. Writing and Running Your First Appium Test	17
7.1 Complete Project Structure	17
7.2 Maven Configuration (pom.xml)	18
7.3 Driver Factory Utility	19
7.4 Base Test Class	20
7.5 Page Object Class	21
7.6 Test Class	27
7.7 TestNG Configuration	28
7.8 Running the Tests	29
7.9 Real-World Project: My SLT App Automation	31
8. Project Structure and Best Practices	32
8.1 Recommended Project Organization	32
8.2 Best Practices	32



09. Challenges Faced and Solutions.....	33
10. Advantages of Appium	34
11. Limitations of Appium	35
12. Conclusion	36
13. References.....	37

1. Introduction to Appium

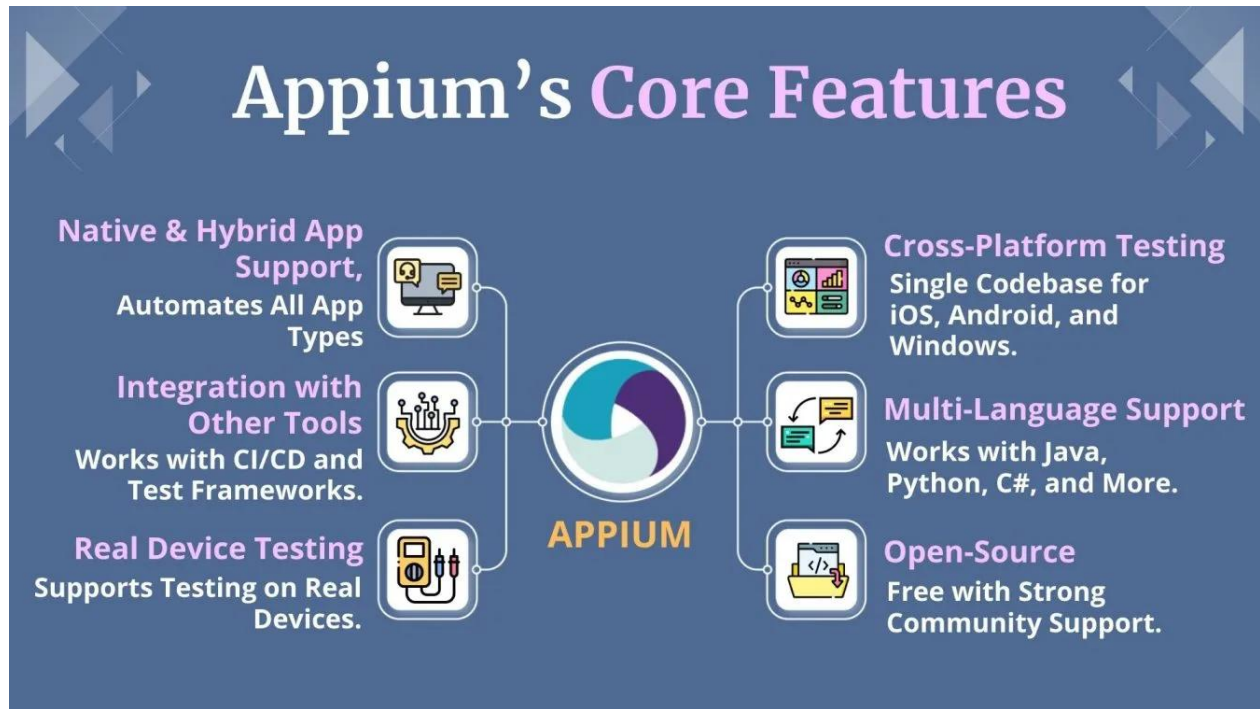
Appium is an open-source test automation framework that has revolutionized mobile application testing. Imagine being able to control your mobile phone programmatically. Tapping buttons, entering text, swiping screens. All through code. That's exactly what Appium enables. It serves as a bridge between your test scripts and mobile devices, allowing comprehensive automation of user interactions.



Think of Appium as **Selenium for mobile apps** - just like Selenium automates web browsers, Appium automates mobile applications. Whether you're testing a banking app, social media platform, or e-commerce application, Appium provides the tools to ensure your app works perfectly across different devices and platforms.



2. What is Appium?



App Type	Description	Examples
Native Apps	Built specifically for a platform using platform SDKs	Camera, Contacts, Gallery apps
Hybrid Apps	Combination of native and web components	Instagram, Facebook
Mobile Web Apps	Websites accessed through mobile browsers	Mobile versions of websites

3. Why Use Appium?

Key Benefits:

1. No App Modification Required

- You don't need to compile or modify your application
- Works with production-ready apps directly from app stores

2. Cross-Platform Testing

- Write test scripts once, run on both Android and iOS
- Significant time and effort savings

3. Programming Language Flexibility

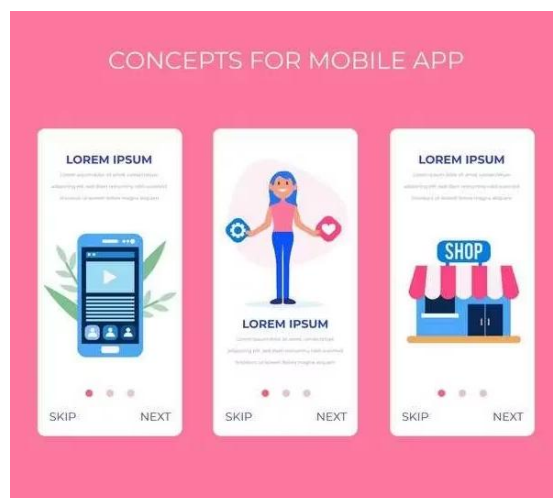
- Use your team's existing programming skills
- No need to learn platform-specific languages

4. Real Device and Emulator Support

- Test on actual hardware for real-world scenarios
- Use emulators for cost-effective testing

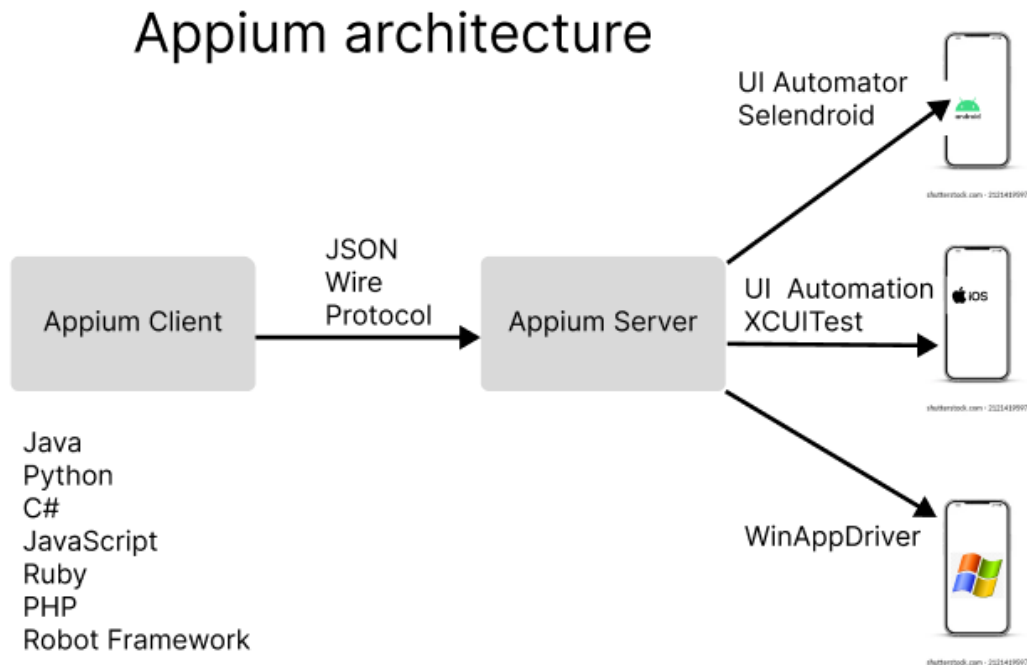
5. CI/CD Integration

- Seamlessly integrates with Jenkins, GitLab CI, GitHub Actions
- Enables continuous testing in DevOps pipelines.



4. Appium Architecture

Detailed Workflow:



Component Breakdown:

4.1 Appium Client

- Your test code written in the preferred programming language
- Uses the WebDriver library to send commands
- Example: Java code using Android Driver class

4.2 Appium Server

- Written in Node.js
- Listens for commands on port 4723 (default)
- Translates WebDriver commands to platform-specific actions

4.3 Vendor Drivers

- **UIAutomator2**: For Android automation
- **XCUITest**: For iOS automation
- **WinAppDriver**: For Windows applications

4.4 Desired Capabilities

JSON configuration that tells Appium:

- Which device to use
- Which app to test
- Platform information
- Automation settings

5. Tools and Technologies

Complete Tool Stack:

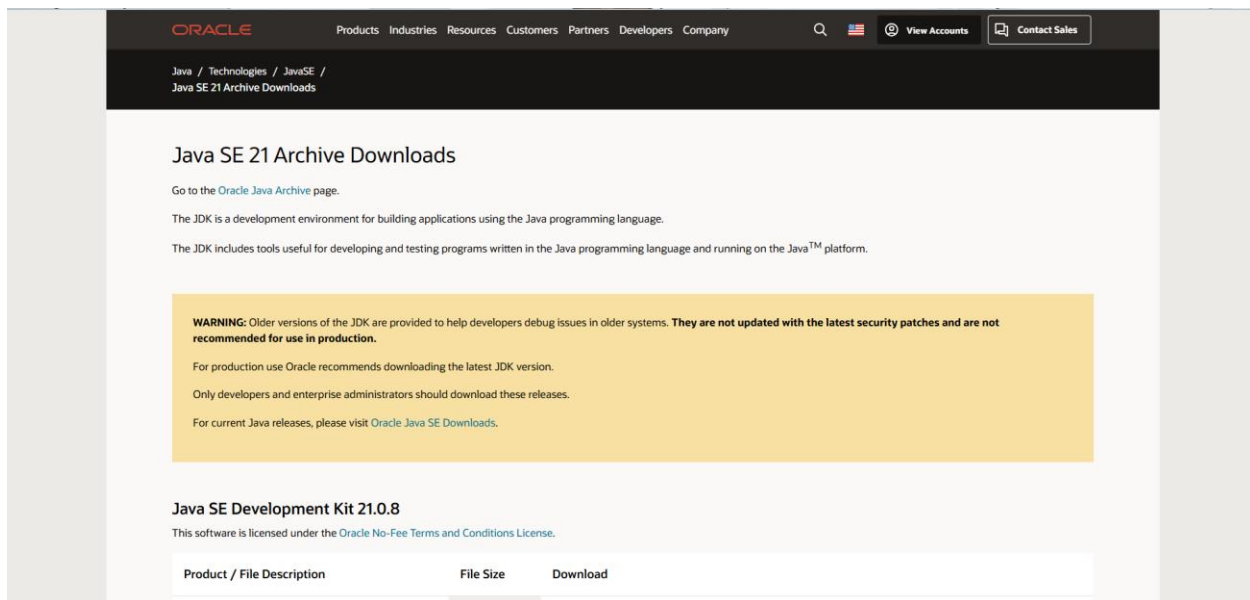
Tool	Version	Purpose	Download Link
Java JDK	17 or 21	Runtime for test execution	Oracle JDK
Node.js	18+	Runtime for Appium server	Node.js
Appium Server	2.19.0+	Main automation engine	npm install -g appium
Appium Inspector	Latest	Element inspection tool	GitHub Releases
Android Studio	Latest	Emulator and SDK management	Android Studio
Maven	3.9+	Dependency management	Maven
IntelliJ IDEA	Community Edition	IDE for test development	IntelliJ

6. Step-by-Step Setup and Installation

6.1 Java Installation (Detailed)

1. Download JDK 17

- Visit Oracle JDK downloads page



- Select Windows x64 Installer
- Run the installer

2. Set Environment Variables

JAVA_HOME = C:\Program Files\Java\jdk-17

Add to PATH: %JAVA_HOME%\bin

3. Verification

```
C:\Windows\System32\cmd.e X + v
Microsoft Windows [Version 10.0.26200.7171]
(c) Microsoft Corporation. All rights reserved.

C:\Sasini>java --version
java 21.0.7 2025-04-15 LTS
Java(TM) SE Runtime Environment (build 21.0.7+8-LTS-245)
Java HotSpot(TM) 64-Bit Server VM (build 21.0.7+8-LTS-245, mixed mode, sharing)
```

Screenshot: Command prompt window showing successful Java version verification using java -version command, confirming proper installation and environment variable setup.



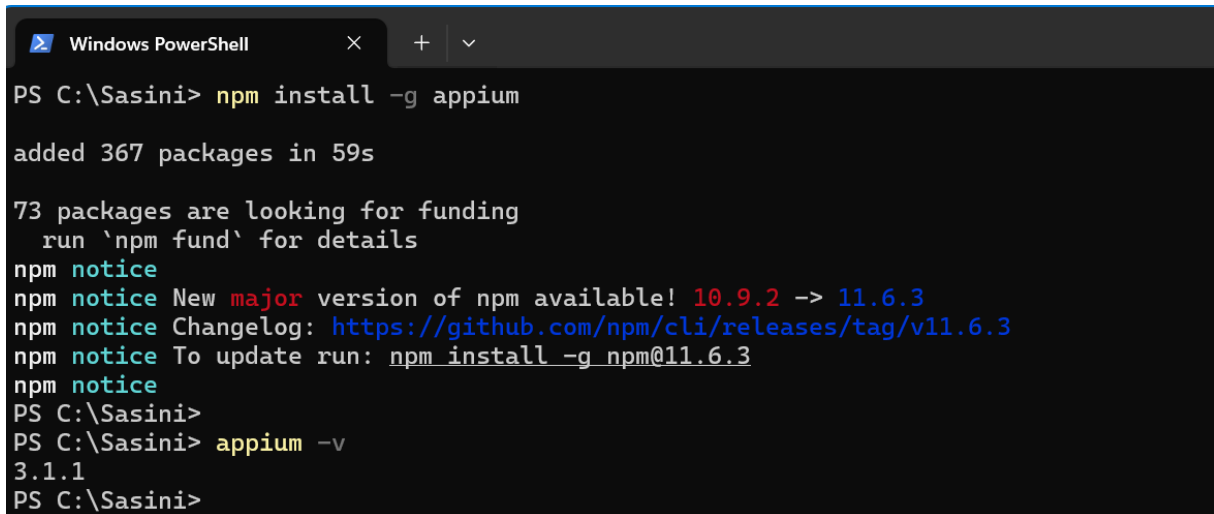
6.2 Node.js and Appium Installation

1. Install Node.js

```
Windows PowerShell X + v
PS C:\Sasini> node -v
v22.14.0
PS C:\Sasini> npm -v
10.9.2
PS C:\Sasini> |
```

Screenshot: Node.js official download page showing the selection of LTS (Long Term Support) version for stable Appium server operation.

2. Install Appium Server



```
Windows PowerShell
PS C:\Sasini> npm install -g appium

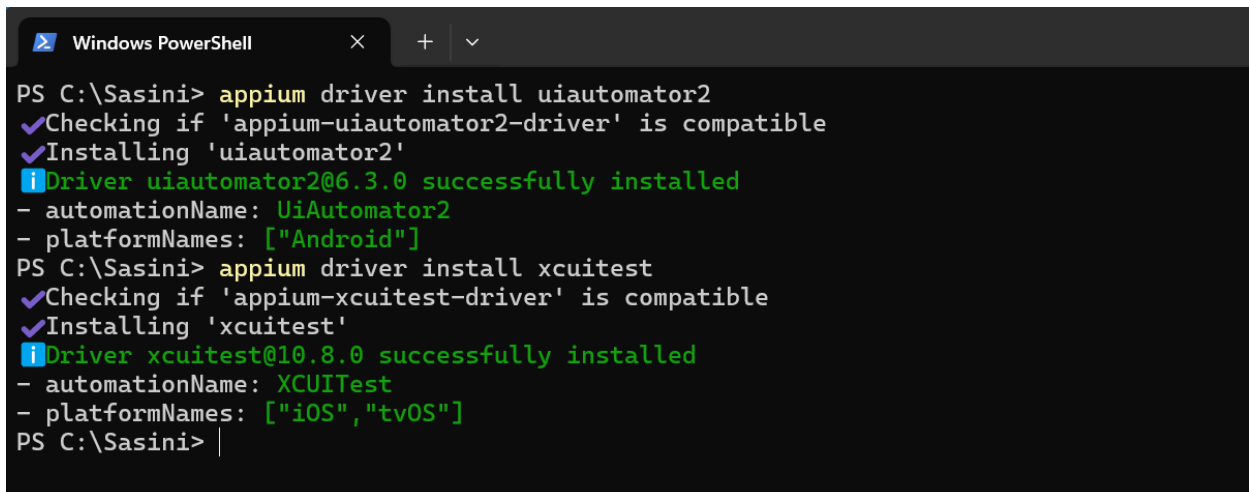
added 367 packages in 59s

73 packages are looking for funding
  run `npm fund` for details

npm notice
npm notice New major version of npm available! 10.9.2 -> 11.6.3
npm notice Changelog: https://github.com/npm/cli/releases/tag/v11.6.3
npm notice To update run: npm install -g npm@11.6.3
npm notice
PS C:\Sasini>
PS C:\Sasini> appium -v
3.1.1
PS C:\Sasini>
```

Screenshot: Terminal/Command Prompt showing the npm installation command for Appium and successful installation confirmation message.

3. Install Appium Drivers



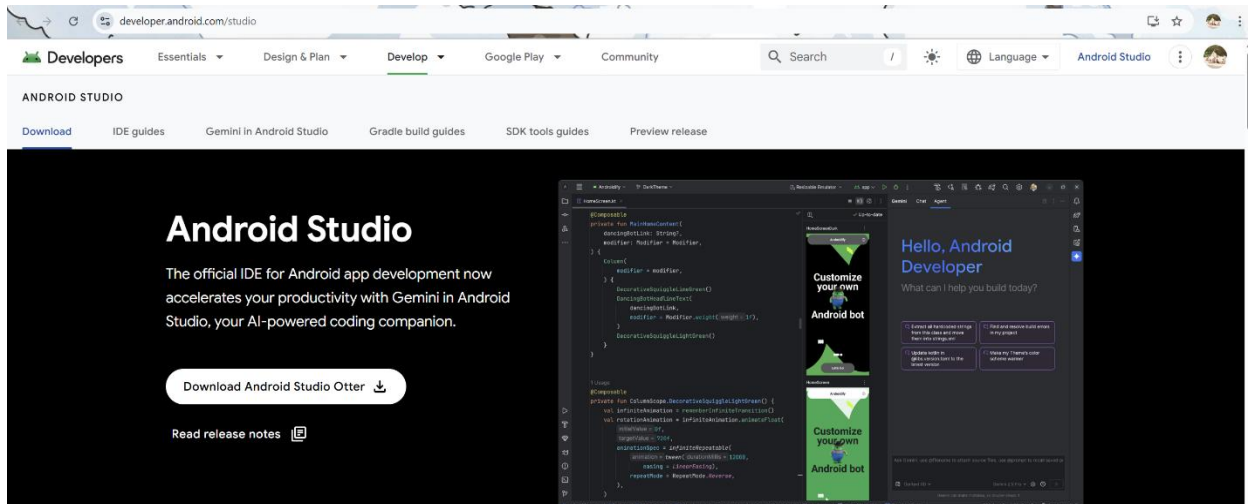
```
Windows PowerShell
PS C:\Sasini> appium driver install uiautomator2
✓Checking if 'appium-uiautomator2-driver' is compatible
✓Installing 'uiautomator2'
iDriver uiautomator2@6.3.0 successfully installed
- automationName: UiAutomator2
- platformNames: ["Android"]
PS C:\Sasini> appium driver install xcuitest
✓Checking if 'appium-xcuitest-driver' is compatible
✓Installing 'xcuitest'
iDriver xcuitest@10.8.0 successfully installed
- automationName: XCUITest
- platformNames: ["iOS","tvOS"]
PS C:\Sasini> |
```

Screenshot: Appium driver installation process showing the command to install UiAutomator2 driver specifically for Android automation capabilities.

6.3 Android Studio Setup

1. Download and Install

- Download Android Studio
- Installing with default components
- Ensure Android SDK, Platform Tools, and Emulator are selected



Screenshot: Appium driver installation process showing the command to install the UIAutomator2 driver specifically for Android automation capabilities.

1. Create Virtual Device

- Open Android Studio → Tools → Device Manager
- Click "Create Device."
- Select Pixel 6 (or any preferred device)
- Download API level 33 (Android 13)
- Configure device settings and finish



Screenshot: Android Virtual Device (AVD) Manager interface showing the device creation wizard with various device profiles and system images available for selection.

2. Start Emulator

```
Windows PowerShell x Windows PowerShell
PS C:\sasini> adb devices
List of devices attached
emulator-5554    device

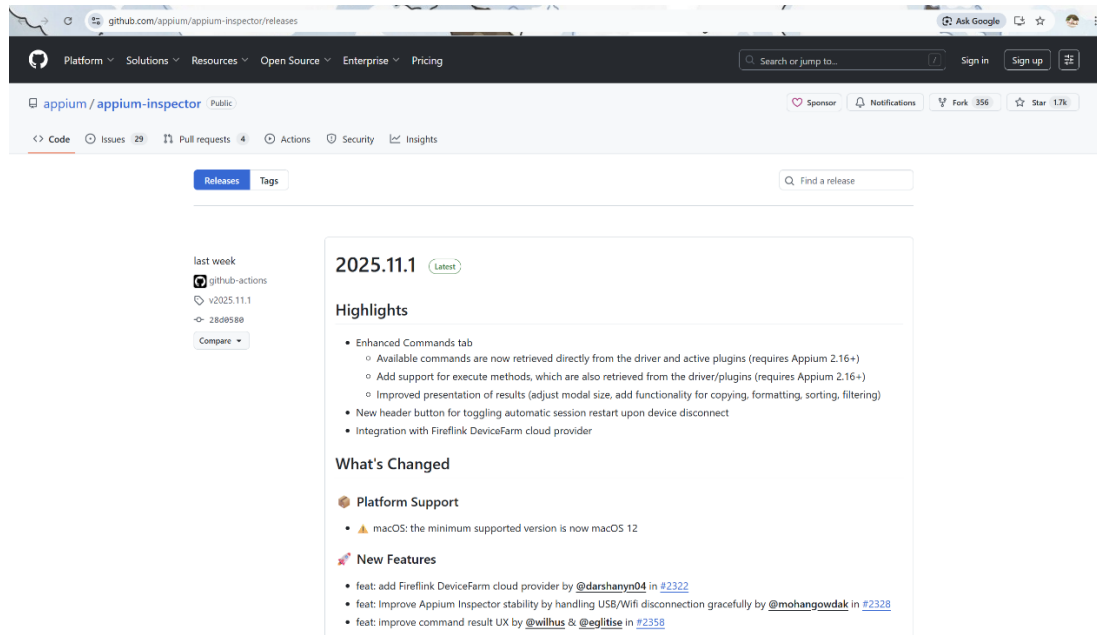
PS C:\sasini> |
```

Screenshot: Android emulator running successfully with the home screen visible, demonstrating the virtual device properly configured and operational.

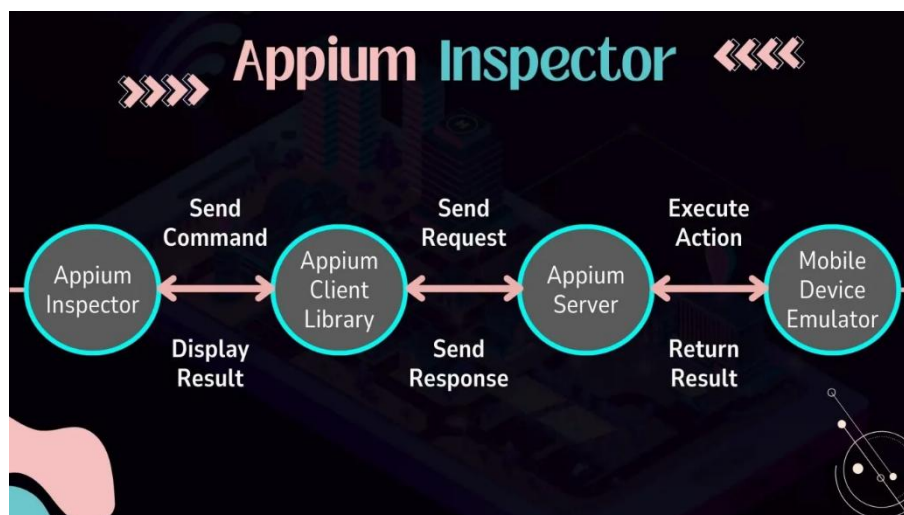
6.4 Appium Inspector Setup

1. Download and Install

- Download from GitHub releases
- Install on your system
- Launch Appium Inspector

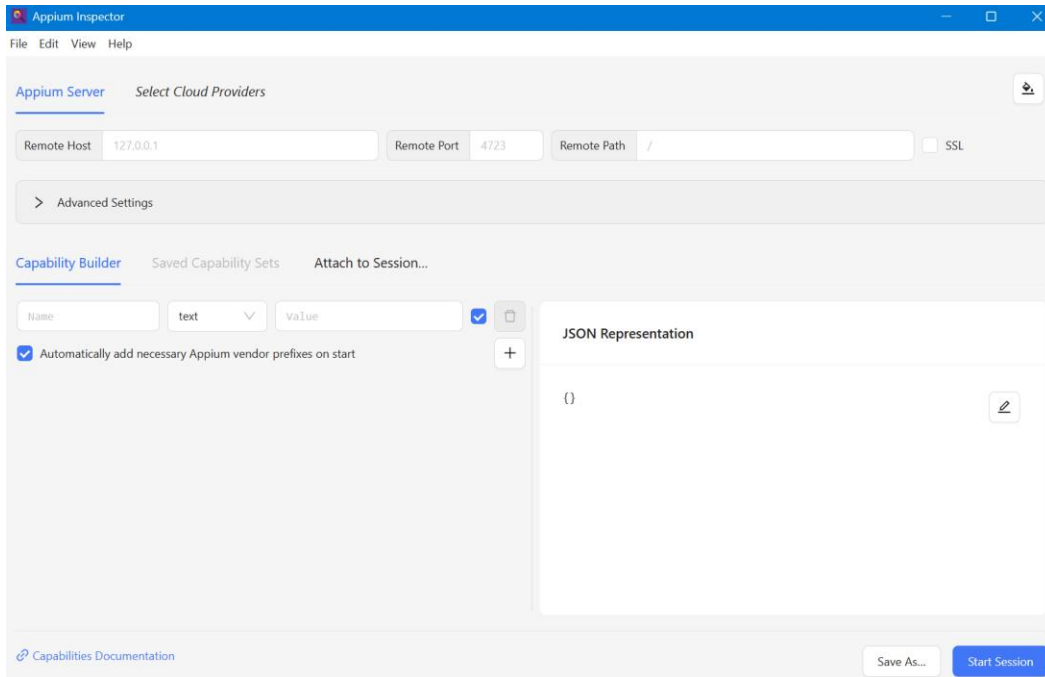


Screenshot: Appium Inspector GitHub releases page showing the available versions for download across different operating systems.



2. Basic Configuration

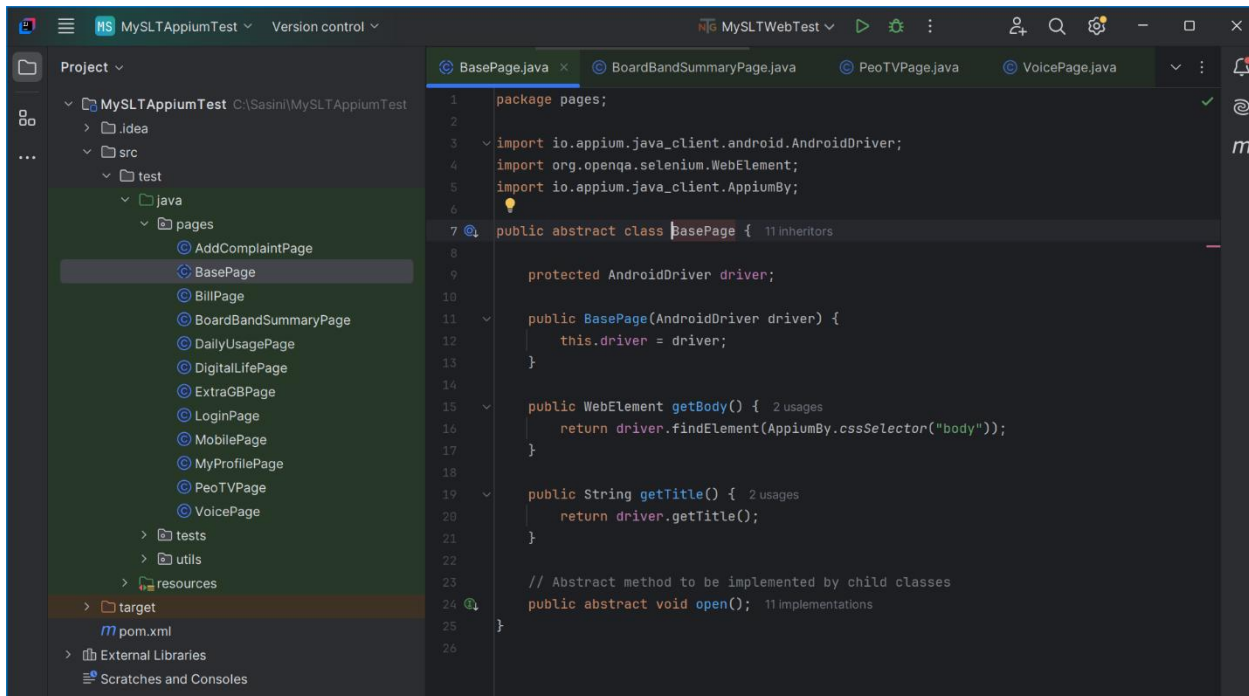
- Host: 127.0.0.1
- Port: 4723
- Path: /



Screenshot: Appium Inspector main interface displaying the connection settings panel, where users configure the server details and desired capabilities for session creation.

7. Writing and Running Your First Appium Test

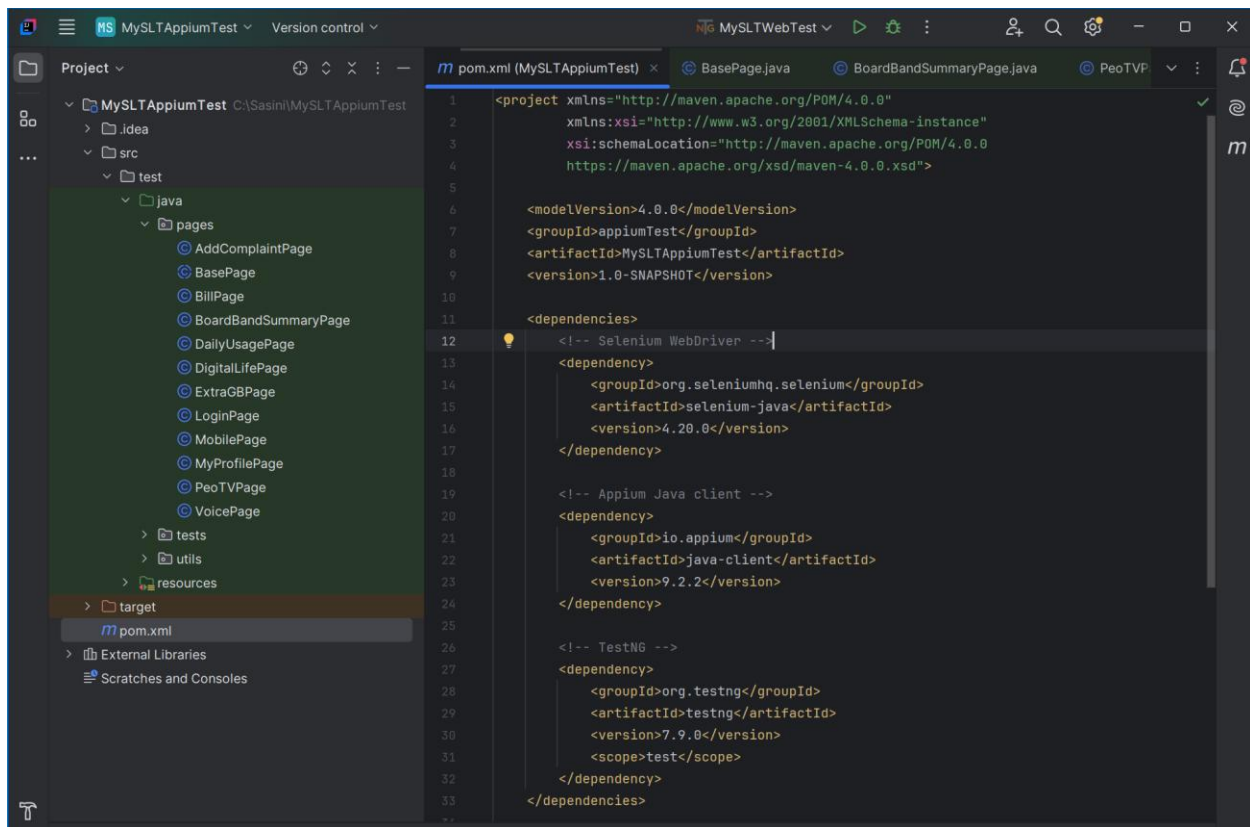
7.1 Complete Project Structure



MySLT Project Structure - IntelliJ IDEA project window showing the complete MySLTAppiumTest project structure with all 12-page classes organized in packages, including pages, tests, utils, and resources folders following Page Object Model best practices.

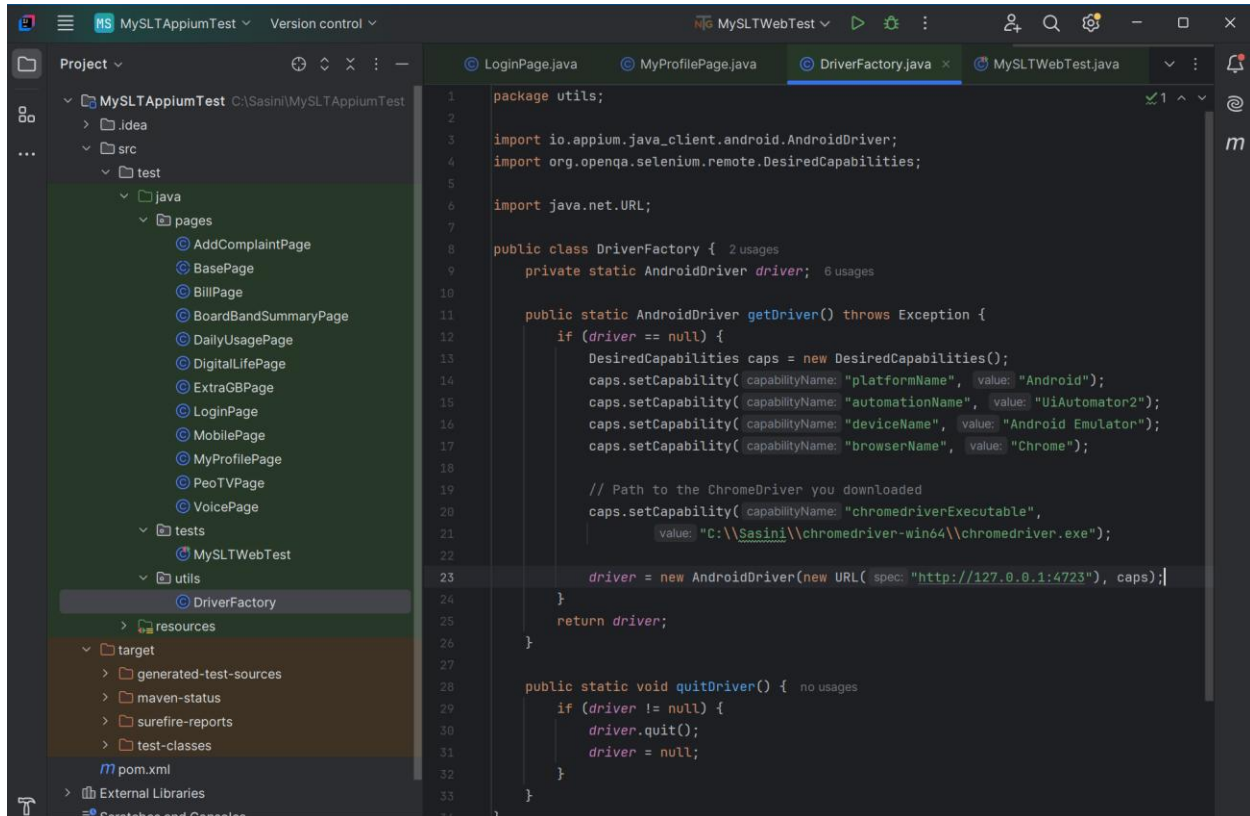


7.2 Maven Configuration (pom.xml)



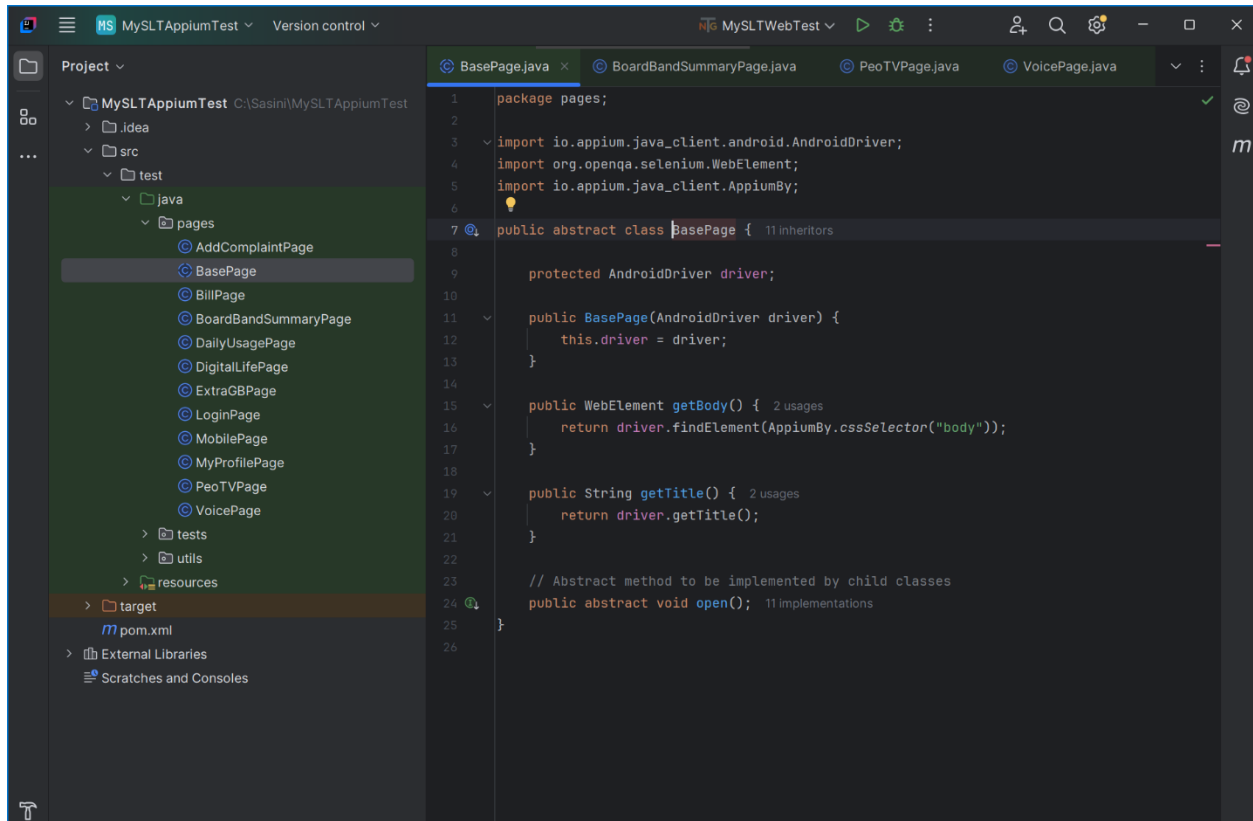
MySLT Maven Configuration - pom.xml file for MySLTAppiumTest project showing essential dependencies, including Appium Java Client 9.2.2, Selenium Java 4.20.0, and TestNG 7.9.0, that form the foundation of the test automation framework

7.3 Driver Factory Utility



MySLT Driver Factory - DriverFactory.java implementation for MySLT project showing Android driver initialization with desired capabilities including platformName, automationName, browserName, and specific ChromeDriver executable path for mobile web automation.

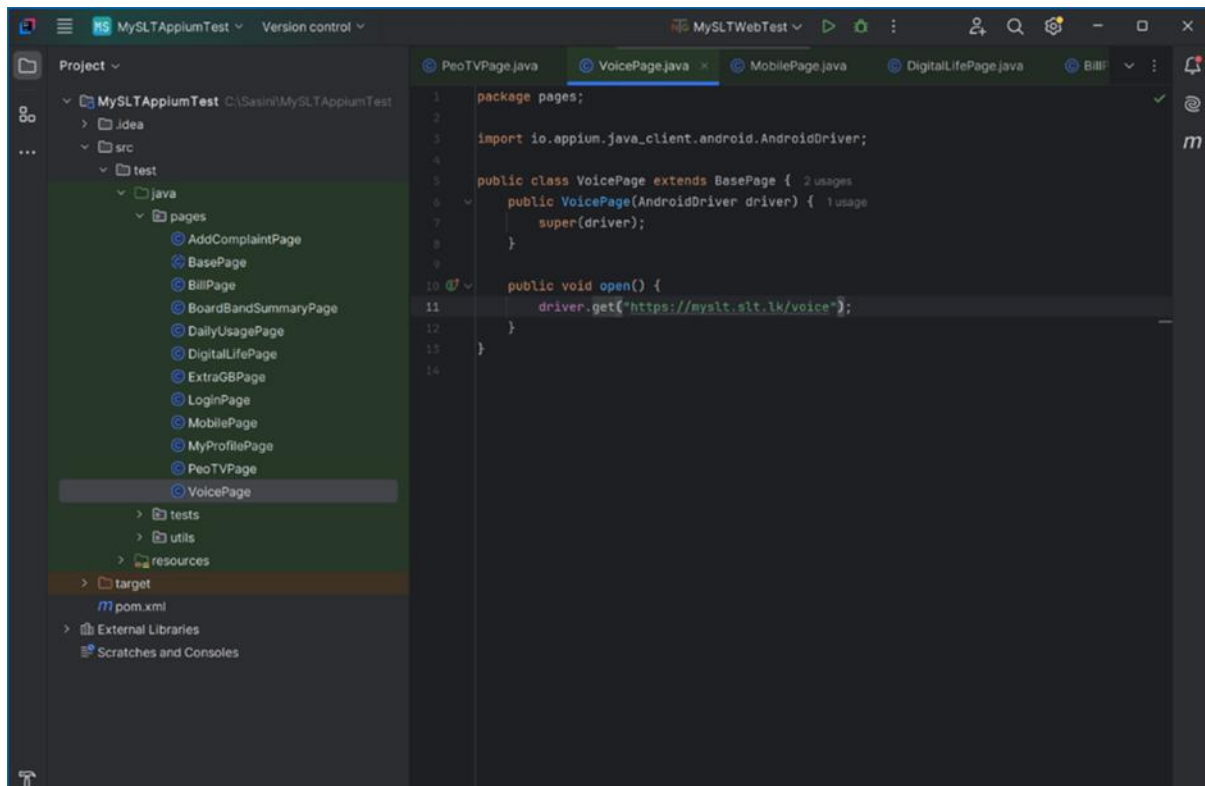
7.4 Base Test Class

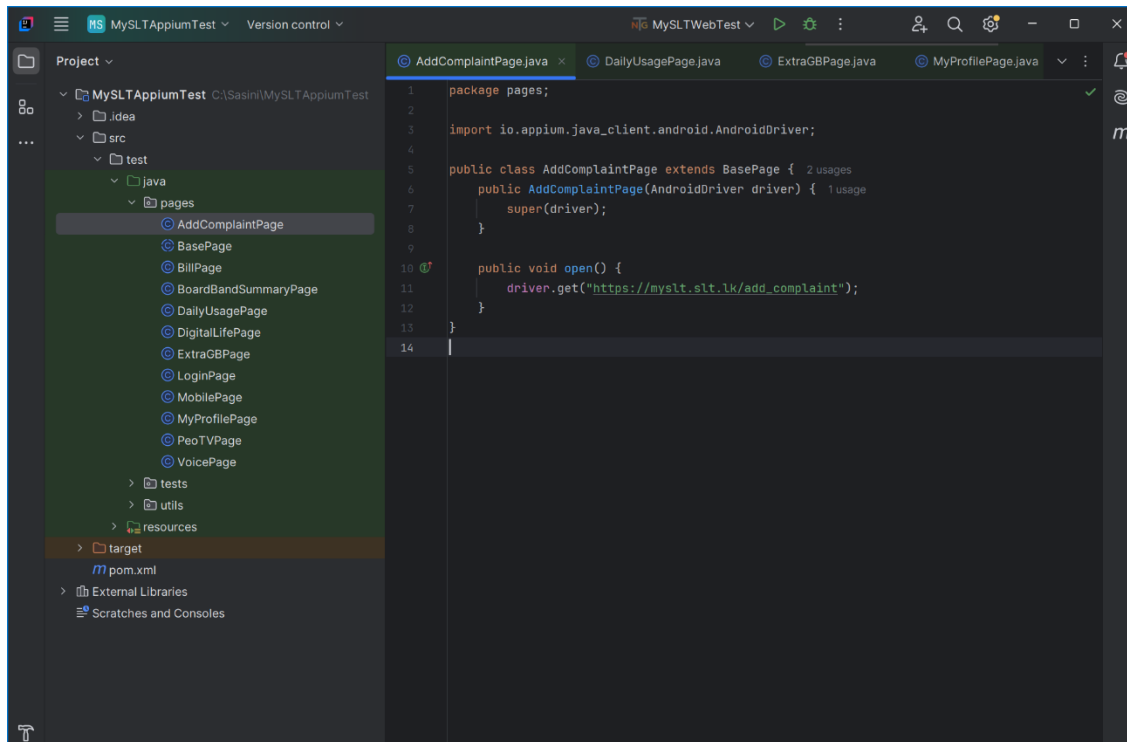


MySLT BasePage Implementation - BasePage.java abstract class serving as the foundation for all page objects in the MySLT project, containing common methods like `getBody()` and `getTitle()`, with an abstract `open ()` method implemented by all child page classes.

7.5 Page Object Class

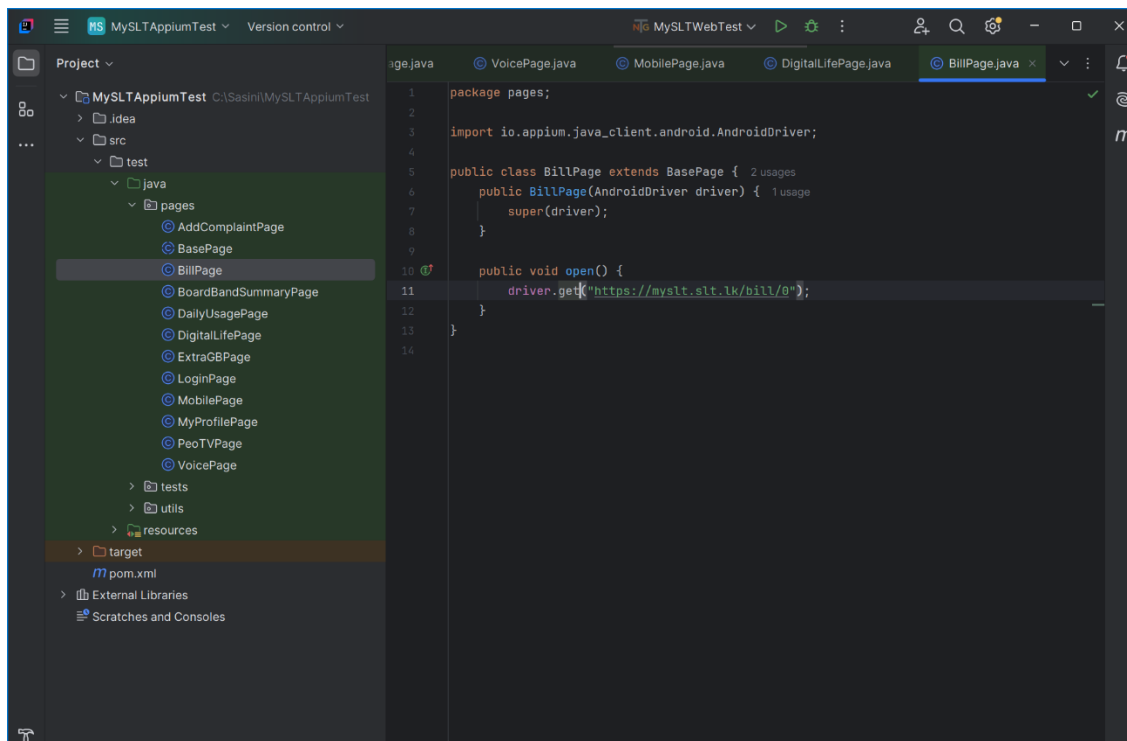
Page Object Classes create separate Java classes for each My SLT page to handle navigation and element interaction, making tests cleaner and easier to maintain. All 12 page classes extend a common BasePage class for shared functionality, allowing centralized updates and reusable code across tests. This approach separates test logic from page details, improving maintainability and scalability.





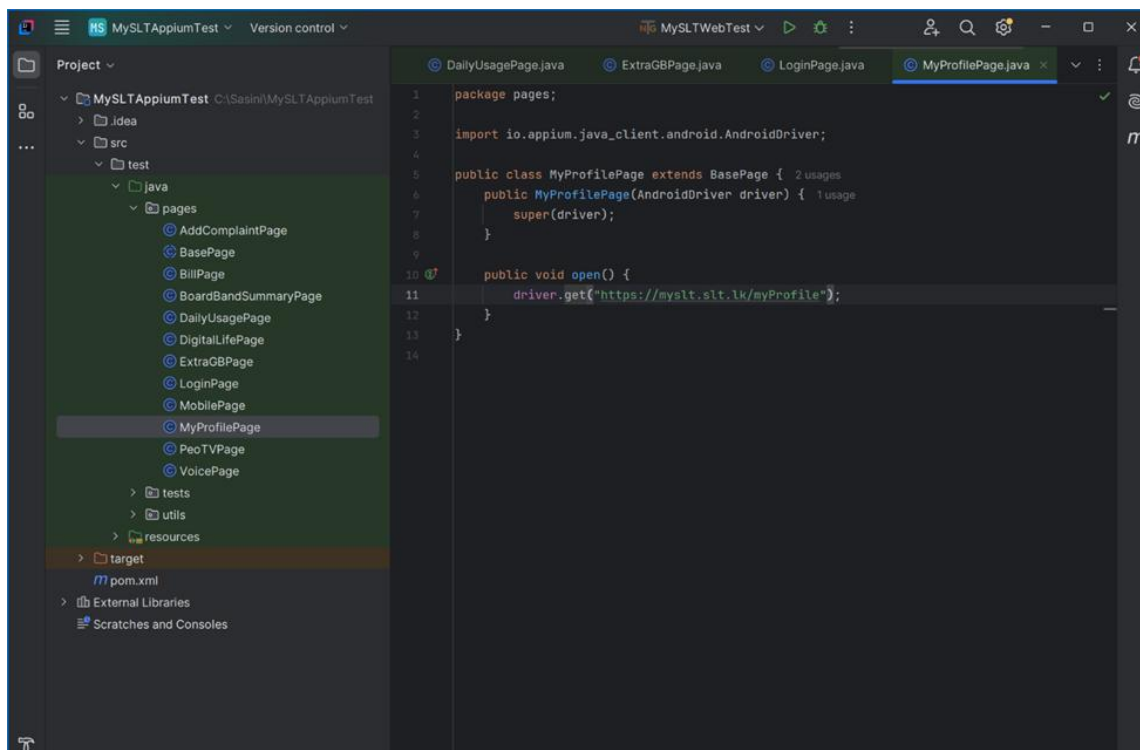
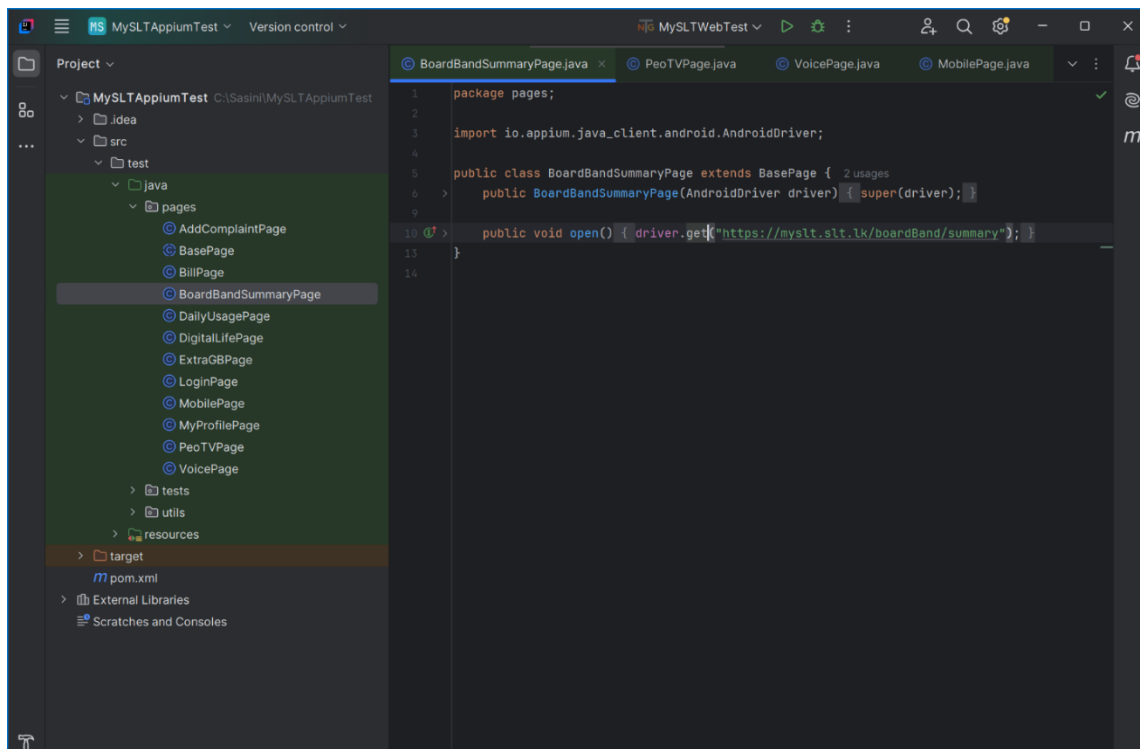
This screenshot shows an IDE window with the project 'MySLTAppiumTest' open. The left sidebar displays the project structure, with the 'pages' folder expanded and 'AddComplaintPage' selected. The main editor area shows the code for 'AddComplaintPage.java'. The code defines a class that extends 'BasePage' and implements an 'open()' method that navigates to a specific URL.

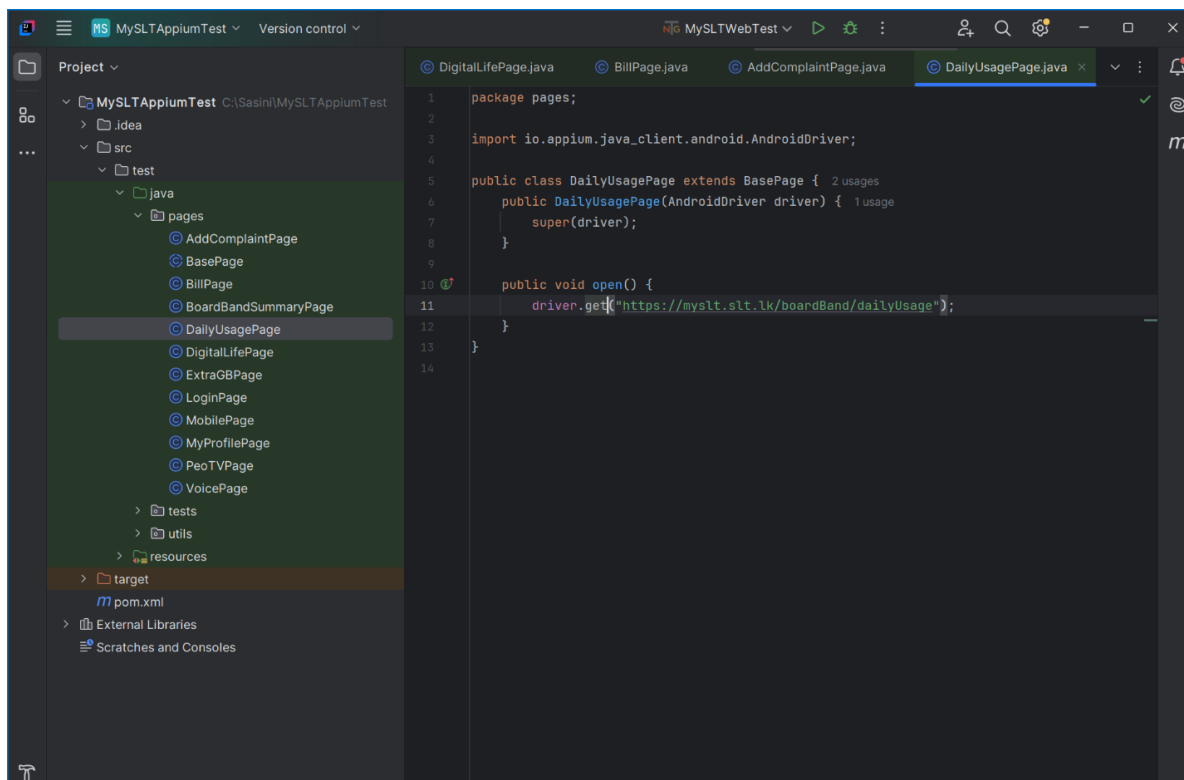
```
1 package pages;
2
3 import io.appium.java_client.android.AndroidDriver;
4
5 public class AddComplaintPage extends BasePage { 2 usages
6     public AddComplaintPage(AndroidDriver driver) { 1 usage
7         super(driver);
8     }
9
10    public void open() {
11        driver.get("https://myslt.slt.lk/add_complaint");
12    }
13 }
14
```



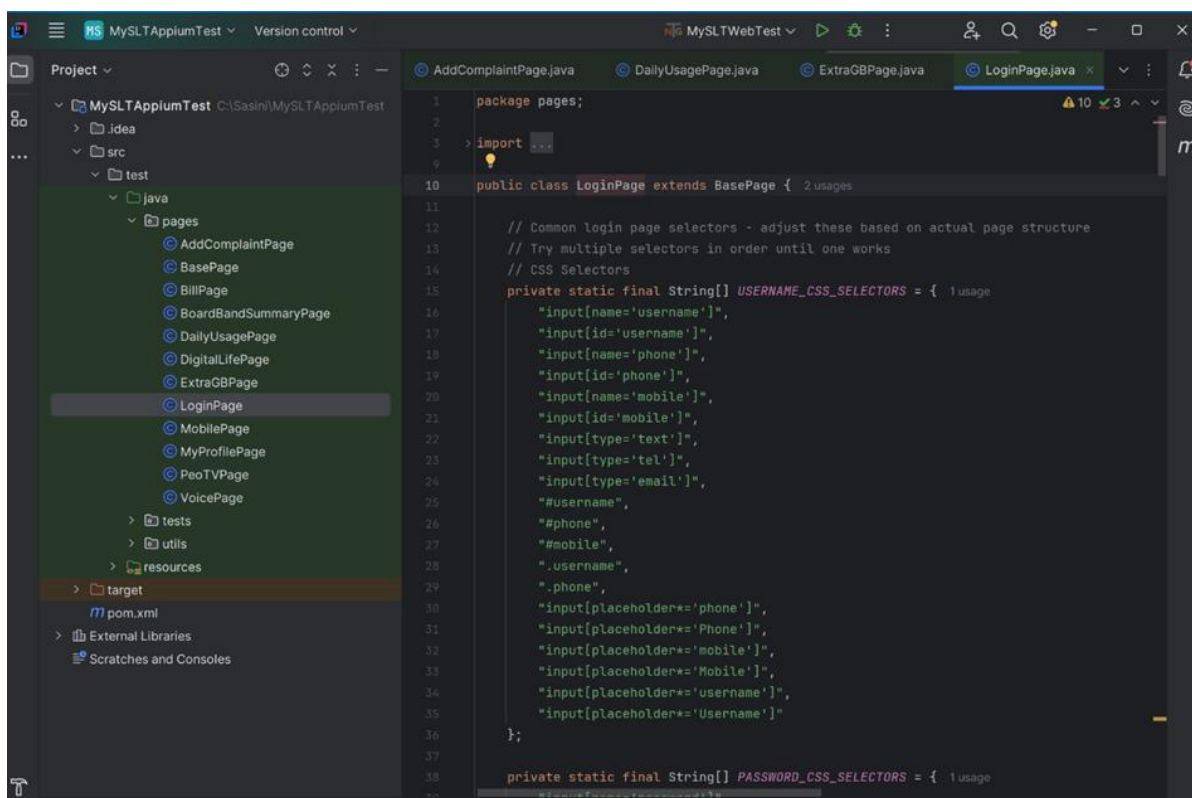
This screenshot shows the same IDE window, but now the 'BillPage.java' file is selected in the sidebar and open in the editor. The code defines a class that extends 'BasePage' and implements an 'open()' method that navigates to a specific URL.

```
1 package pages;
2
3 import io.appium.java_client.android.AndroidDriver;
4
5 public class BillPage extends BasePage { 2 usages
6     public BillPage(AndroidDriver driver) { 1 usage
7         super(driver);
8     }
9
10    public void open() {
11        driver.get("https://myslt.slt.lk/bill/0");
12    }
13 }
14
```

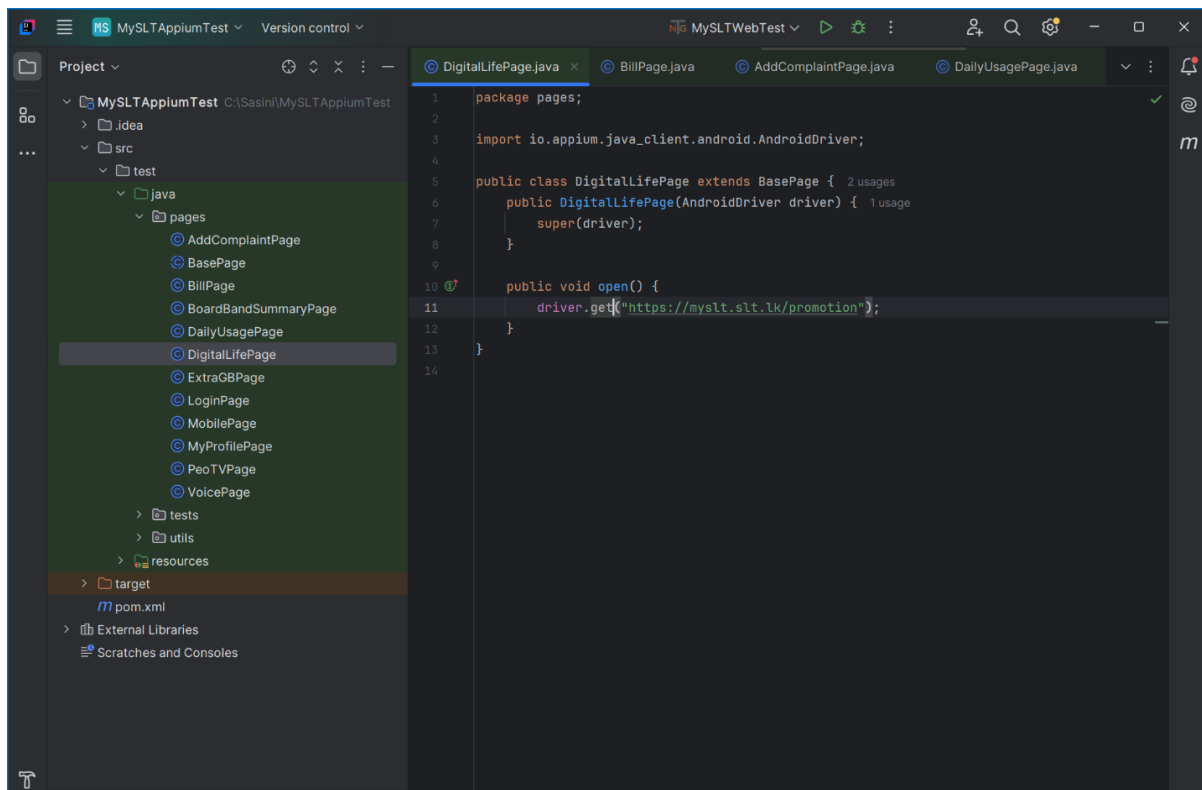
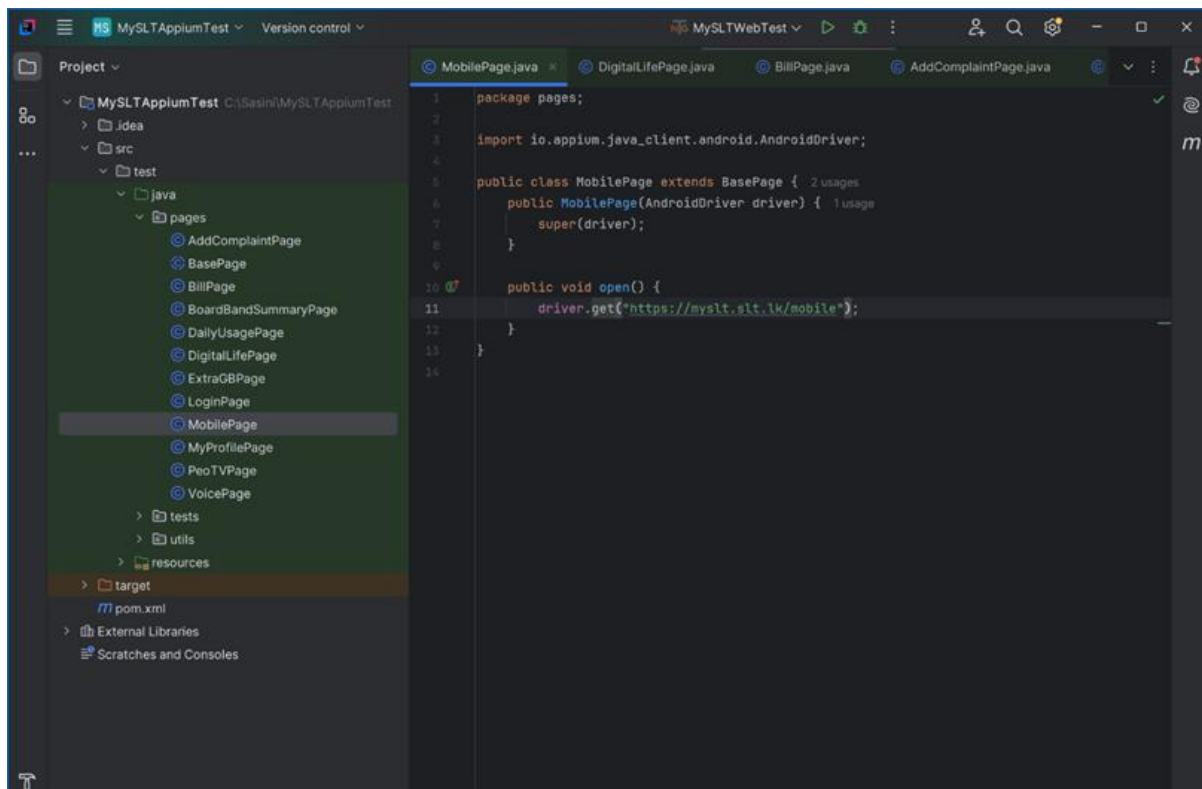




```
1 package pages;
2
3 import io.appium.java_client.android.AndroidDriver;
4
5 public class DailyUsagePage extends BasePage { 2 usages
6     public DailyUsagePage(AndroidDriver driver) { 1 usage
7         super(driver);
8     }
9
10    public void open() {
11        driver.get("https://myslt.slt.lk/boardBand/dailyUsage");
12    }
13 }
14
```



```
1 package pages;
2
3 import io.appium.java_client.android.AndroidDriver;
4
5 public class LoginPage extends BasePage { 2 usages
6     public LoginPage(AndroidDriver driver) { 1 usage
7         super(driver);
8     }
9
10    public void open() {
11        driver.get("https://myslt.slt.lk/boardBand/dailyUsage");
12    }
13
14    // Common login page selectors - adjust these based on actual page structure
15    // Try multiple selectors in order until one works
16    // CSS Selectors
17    private static final String[] USERNAME_CSS_SELECTORS = { 1 usage
18        "input[name='username']",
19        "input[id='username']",
20        "input[name='phone']",
21        "input[id='phone']",
22        "input[name='mobile']",
23        "input[id='mobile']",
24        "input[type='text']",
25        "input[type='tel']",
26        "input[type='email']",
27        "#username",
28        "#phone",
29        "#mobile",
30        ".username",
31        ".phone",
32        "input[placeholder='phone']",
33        "input[placeholder='Phone']",
34        "input[placeholder='mobile']",
35        "input[placeholder='Mobile']",
36        "input[placeholder='username']",
37        "input[placeholder='Username']"
38    };
39
40    private static final String[] PASSWORD_CSS_SELECTORS = { 1 usage
41        "input[name='password']",
42        "input[id='password']",
43        "input[type='password']",
44        "#password",
45        ".password"
46    };
47 }
48
```

```

1 package pages;
2
3 import io.appium.java_client.android.AndroidDriver;
4
5 public class ExtraGBPage extends BasePage { 2 usages
6     public ExtraGBPage(AndroidDriver driver) { 1 usage
7         super(driver);
8     }
9
10    public void open() {
11        driver.get("https://myslt.slt.lk/boardBand/summary/extraGB");
12    }
13 }
14

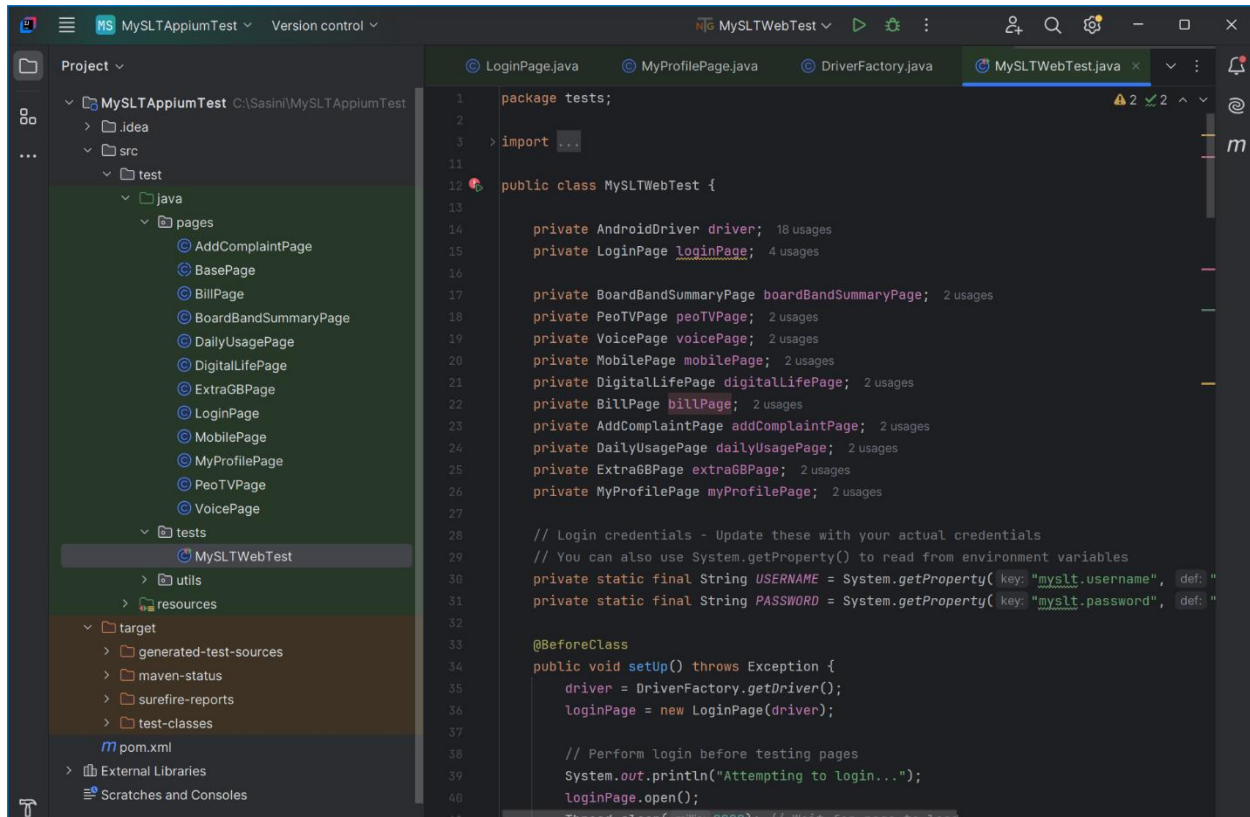
```

```

1 package pages;
2
3 import io.appium.java_client.android.AndroidDriver;
4
5 public class PeoTVPage extends BasePage { 2 usages
6     public PeoTVPage(AndroidDriver driver) { super(driver); }
7
8
9
10    public void open() {
11        driver.get("https://myslt.slt.lk/peoTV");
12    }
13 }
14

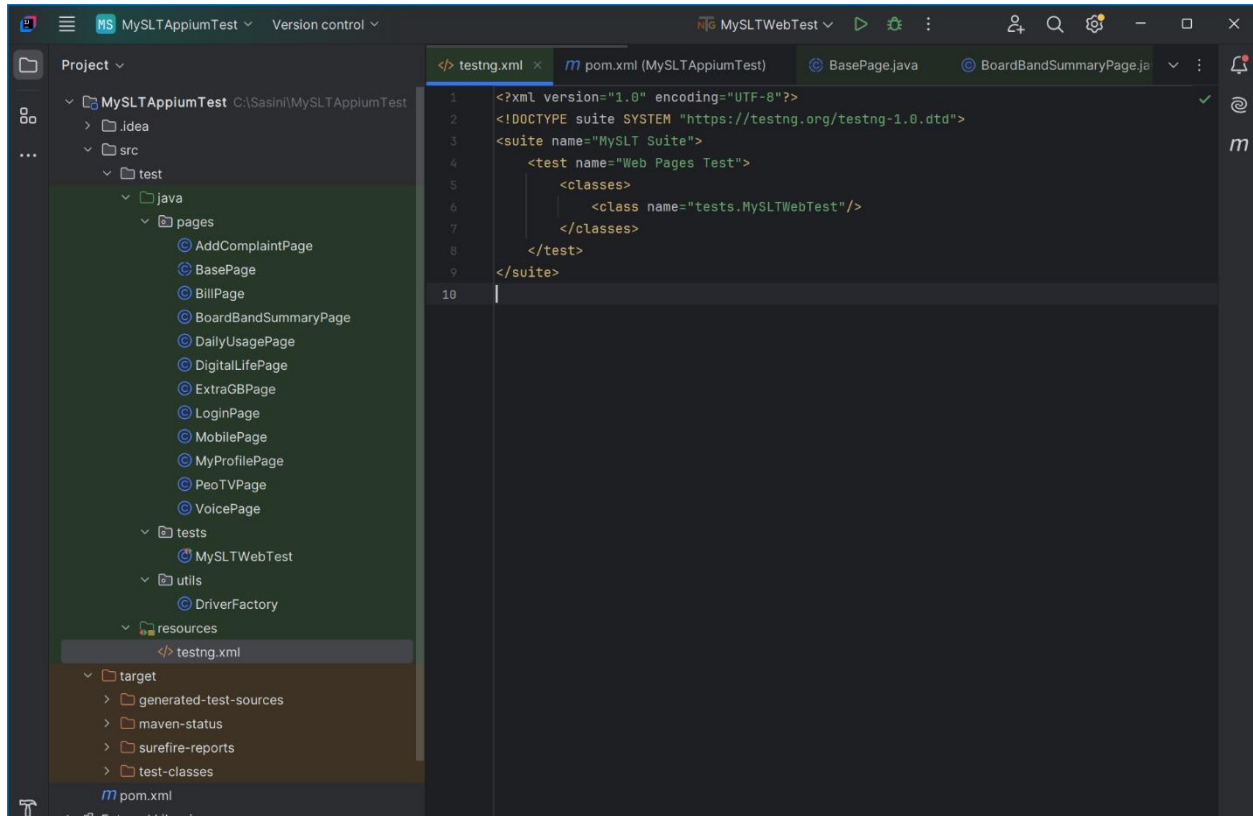
```

7.6 Test Class



MySLT Test Class - *MySLTWebTest.java* showing the complete test implementation with all 12-page object declarations, login credential configuration, and TestNG annotations for setup, execution, and teardown of the MySLT application tests.

7.7 TestNG Configuration

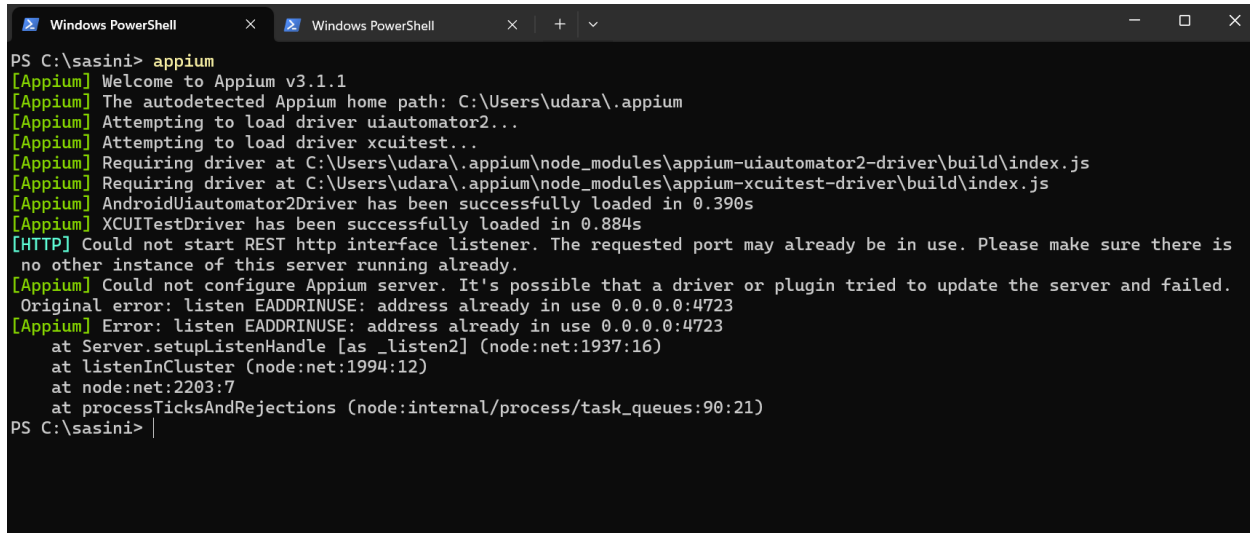


MySLT Test Execution Results - Terminal output showing successful test execution with "BUILD SUCCESS", all tests passed (Tests run: 1, Failures: 0, Errors: 0, Skipped: 0), and confirmation of MySLT page loading with welcome message.

7.8 Running the Tests

Step-by-Step Execution:

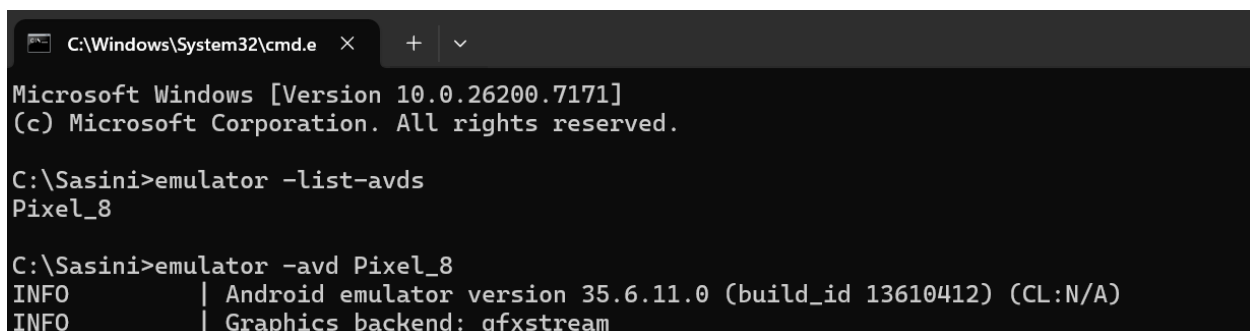
1. Start Appium Server:



```
PS C:\sasini> appium
[Appium] Welcome to Appium v3.1.1
[Appium] The autodetected Appium home path: C:\Users\udara\.appium
[Appium] Attempting to load driver uiautomator2...
[Appium] Attempting to load driver xcuitest...
[Appium] Requiring driver at C:\Users\udara\.appium\node_modules\appium-uiautomator2-driver\build\index.js
[Appium] Requiring driver at C:\Users\udara\.appium\node_modules\appium-xcuitest-driver\build\index.js
[Appium] AndroidUiautomator2Driver has been successfully loaded in 0.390s
[Appium] XCUIETestDriver has been successfully loaded in 0.884s
[HTTP] Could not start REST http interface listener. The requested port may already be in use. Please make sure there is
no other instance of this server running already.
[Appium] Could not configure Appium server. It's possible that a driver or plugin tried to update the server and failed.
Original error: listen EADDRINUSE: address already in use 0.0.0.0:4723
[Appium] Error: listen EADDRINUSE: address already in use 0.0.0.0:4723
    at Server.setupListenHandle [as _listen2] (node:net:1937:16)
    at listenInCluster (node:net:1994:12)
    at node:net:2203:7
    at processTicksAndRejections (node:internal/process/task_queues:90:21)
PS C:\sasini>
```

Screenshot: Terminal showing Appium server successfully started on port 4723

2. Start Android Emulator:



```
C:\Windows\System32\cmd.e
Microsoft Windows [Version 10.0.26200.7171]
(c) Microsoft Corporation. All rights reserved.

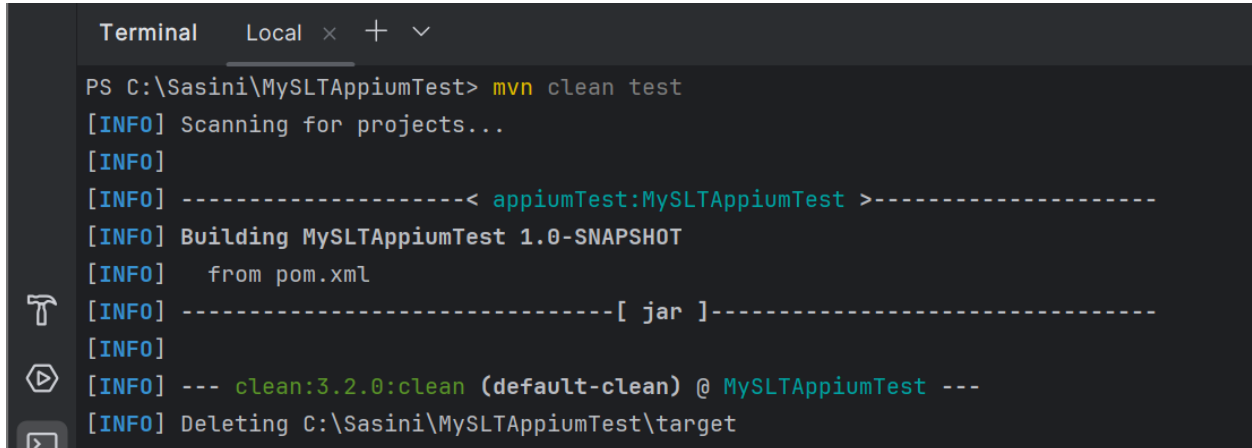
C:\Sasini>emulator -list-avds
Pixel_8

C:\Sasini>emulator -avd Pixel_8
INFO          | Android emulator version 35.6.11.0 (build_id 13610412) (CL:N/A)
INFO          | Graphics backend: gfxstream
```

Screenshot: Android emulator running with the app launched

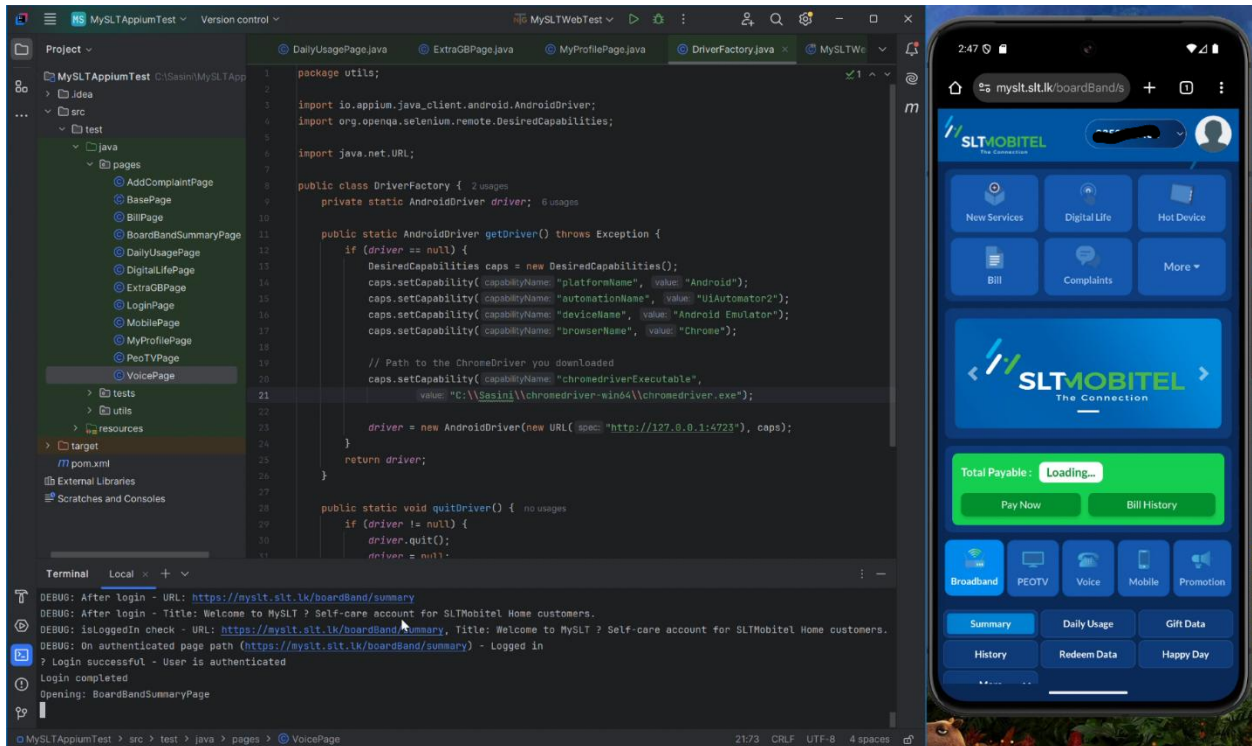
1. Run Tests from IntelliJ:

- Right-click on testng.xml → Run
- Or run from command line: mvn clean test



```
Terminal Local x + v
PS C:\Sasini\MySLTAppiumTest> mvn clean test
[INFO] Scanning for projects...
[INFO]
[INFO] -----< appiumTest:MySLTAppiumTest >-----
[INFO] Building MySLTAppiumTest 1.0-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- clean:3.2.0:clean (default-clean) @ MySLTAppiumTest ---
[INFO] Deleting C:\Sasini\MySLTAppiumTest\target
```

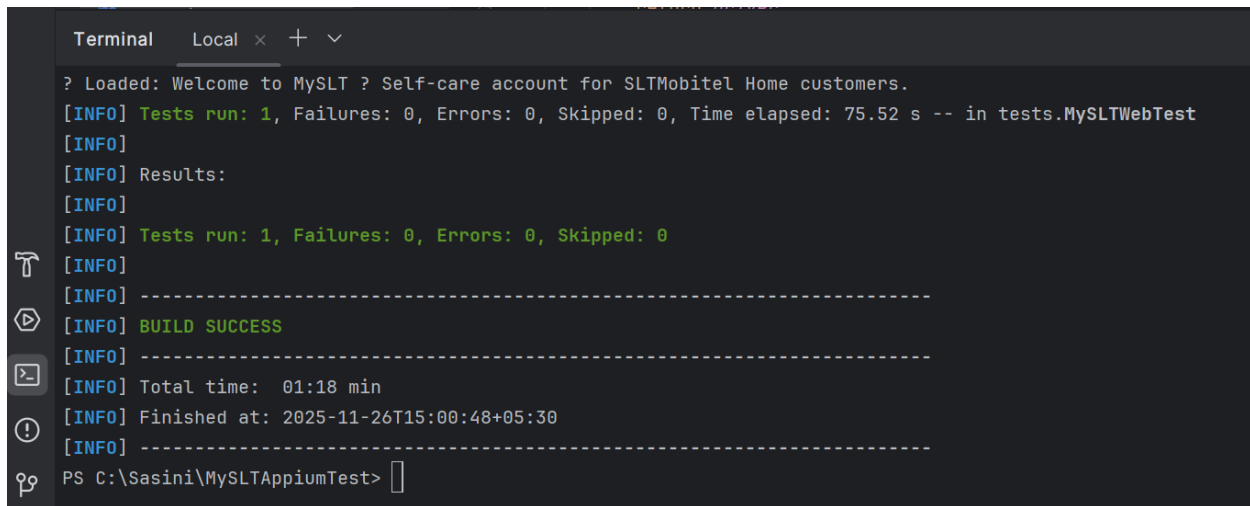
Screenshot: IntelliJ IDEA showing test execution with green passed status



2. Execution:

- Emulator will launch app
- Tests will be executed automatically

- You'll see buttons being pressed automatically
- Results will display in the console



```

Terminal Local x + v
? Loaded: Welcome to MySLT ? Self-care account for SLTMobitel Home customers.
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 75.52 s -- in tests.MySLTWebTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 01:18 min
[INFO] Finished at: 2025-11-26T15:00:48+05:30
[INFO] -----
PS C:\Sasini\MySLTAppiumTest>

```

Screenshot: Console output showing test execution results and success messages

7.9 Real-World Project: My SLT App Automation

This section documents the implementation of a real-world test automation project for the "My SLT" customer web portal.

Pages Automated

1. Broadband Summary (/boardBand/summary)
2. PeoTV Services (/peoTV)
3. Voice Services (/voice)
4. Mobile Services (/mobile)
5. Digital Life (/digitalLife)
6. Bill Information (/bill/0)
7. Add Complaint (/add_complaint)
8. Daily Usage (/boardBand/dailyUsage)
9. Extra GB Management (/boardBand/summary/extraGB)
10. User Profile (/myProfile)

8. Project Structure and Best Practices

8.1 Recommended Project Organization

Text

```
MySLTAppiumTest/
├── pom.xml                # Maven configuration file
├── src/
│   ├── test/
│   │   ├── java/
│   │   │   ├── pages/    # Page Object Model classes
│   │   │   │   ├── BasePage.java # Abstract base page class
│   │   │   │   ├── LoginPage.java # Login page implementation
│   │   │   │   ├── BoardBandSummaryPage.java # Broadband summary page
│   │   │   │   ├── PeoTVPage.java # PeoTV services page
│   │   │   │   ├── VoicePage.java # Voice services page
│   │   │   │   ├── MobilePage.java # Mobile services page
│   │   │   │   ├── DigitalLifePage.java # Digital life page
│   │   │   │   ├── BillPage.java # Bill information page
│   │   │   │   ├── AddComplaintPage.java # Complaint submission page
│   │   │   │   ├── DailyUsagePage.java # Daily usage page
│   │   │   │   ├── ExtraGBPage.java # Extra GB management page
│   │   │   │   └── MyProfilePage.java # User profile page
│   │   │   ├── tests/    # Test classes
│   │   │   │   └── MySLTWebTest.java # Main test class
│   │   │   ├── utils/    # Utility classes
│   │   │   │   └── DriverFactory.java # Driver initialization utility
│   │   ├── resources/    # Configuration files
│   │   │   └── testng.xml # TestNG configuration
│   └── target/           # Compiled output directory
```

8.2 Best Practices

1. **Use Page Object Model:** Separate element locators and actions from test logic
2. **Implement Explicit Waits:** Avoid using `Thread.sleep()`
3. **Use Meaningful Selectors:** Prefer accessibility IDs over XPath when possible
4. **Handle Permissions:** Use `autoGrantPermissions` for Android
5. **Implement Proper Teardown:** Always quit driver after tests
6. **Use Configuration Files:** Externalize desired capabilities

09. Challenges Faced and Solutions

Challenges:

1. **ChromeDriver not found** - Appium couldn't find ChromeDriver file
2. **Login page keeps changing** - The Login page HTML is different each time
3. **12 pages, hard to organize** - Too many page classes to manage
4. **Tests fail randomly** - Tests sometimes pass, sometimes fail
5. **Passwords in code** - Login details visible in source code
6. **Build doesn't work** - Maven build fails with errors
7. **Code repeating everywhere** - Same code copied in all pages

Solutions:

1. **Used exact file path** - Told Appium exactly where ChromeDriver is
2. **Made 15+ selectors** - Created many options to find login fields
3. **Created BasePage class** - Made parent class for all pages
4. **Used proper waits** - Replaced sleep () with better waiting
5. **Made credentials configurable** - Can set passwords outside code
6. **Fixed version compatibility** - Found Appium/Selenium versions that work
7. **Used inheritance** - All pages share code from BasePage



10. Advantages of Appium

Technical Advantages:

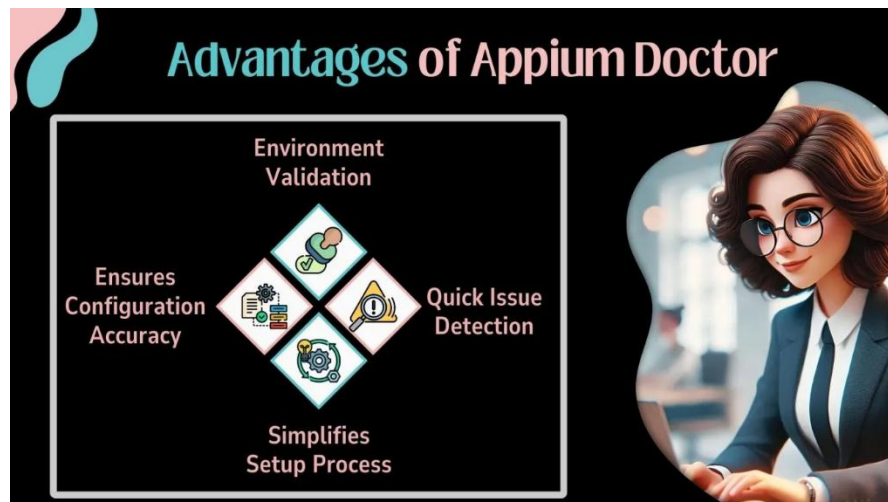
- **True Cross-Platform:** Single API for Android and iOS
- **No Source Code Required:** Test production apps directly
- **Web Standards:** Uses W3C WebDriver Protocol
- **Extensible Architecture:** Support for custom plugins

Business Advantages:

- **Cost-effective:** Open-source and free
- **Skill Reuse:** Leverage existing Selenium knowledge
- **CI/CD Ready:** Easy integration with DevOps pipelines
- **Community Support:** Active community and regular updates

Testing Advantages:

- **Real Device Testing:** Test on actual hardware
- **Parallel Execution:** Run multiple tests simultaneously
- **Comprehensive Reporting:** Integration with reporting tools
- **Gesture Support:** Supports complex mobile gestures





11. Limitations of Appium

Technical Limitations:

- **Performance:** Slower than native automation tools
- **Complex Setup:** Multiple components to configure
- **Limited Gesture Support:** Some advanced gestures may not be supported
- **No Image Recognition:** Cannot identify elements by images

Platform Limitations:

- **iOS Restrictions:** Requires additional setup on macOS
- **Android Fragmentation:** Testing across many device types can be challenging
- **Windows Support:** Limited compared to Android/iOS

Operational Limitations:

- **Maintenance Overhead:** Requires updates for new OS versions
- **Learning Curve:** Steep initial learning curve for beginners
- **Flakiness:** Tests can be less stable than native alternatives

12. Conclusion

Appium has established itself as the leading open-source solution for mobile test automation. Through this comprehensive implementation:

Business Value:

Appium enables organizations to:

- **Reduce testing costs** through automation
- **Improve app quality** with comprehensive testing
- **Accelerate release cycles** with continuous testing
- **Ensure cross-platform consistency** across devices

The skills and knowledge gained from this Appium implementation provide a solid foundation for mobile test automation that can be applied to real-world projects across various domains.



13. References

Official Documentation:

- [Appium Official Documentation](#)
- [Selenium WebDriver Documentation](#)
- [TestNG Documentation](#)

Download Links:

- [Appium Server Installation Guide](#)
- [Appium Inspector Downloads](#)
- [Android Studio Download](#)

Learning Resources:

- [Appium Mobile Automation Tutorials](#)
- [Selenium WebDriver Guide](#)
- [Mobile Testing Best Practices](#)

Community Support:

- [Appium Discuss Forum](#)
- [Stack Overflow Appium Tag](#)
- [GitHub Issues](#)